ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 10

Funciones: escribid una vez, usad mil

- ¿Por qué funciones?
- La gran frontera: echo frente a return
- Parámetros: valores por defecto y argumentos con nombre
- Tipos: decirle a PHP qué entra y qué sale
- Ámbito: cada función tiene su habitación
- Funciones flecha: un primer vistazo
- Cómo escribir buenas funciones



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Si alguna vez os pilláis copiando y pegando el mismo trozo de código en tres sitios, parad. Ahí falta una **función**.

Ya escribisteis alguna en la clase 5. Hoy las vemos con calma: la diferencia entre mostrar y devolver, los parámetros, los tipos y por qué las variables de dentro de una función no se ven desde fuera. Con esto vais a escribir código más corto, más ordenado y mucho más fácil de arreglar.

1
🤔 ¿Por qué funciones?



2
⚠️ La gran frontera: echo frente a return



3
📘 Parámetros: valores por defecto y argumentos con nombre



4
👌 Tipos: decirle a PHP qué entra y qué sale



5
🔍 Ámbito: cada función tiene su habitación



6
😮 Funciones flecha: un primer vistazo



7
👍 Cómo escribir buenas funciones



8
🚀 Ejercicios



9
👋 Lo que tenéis que llevaros de hoy





CLASE 10 · PUNTO 1

🤔 ¿Por qué funciones?

Una función es una **receta con nombre**: la escribís una vez y la «cocináis» cada vez que la necesitáis, con los ingredientes que le paséis [1].

Mirad este código. Calcula el precio con IVA de tres productos:

01-por-que-funciones.php — php-paso-a-paso/clase10 — Visual Studio Code

01-por-que-funciones.php

```php
<?php
echo "Camiseta: " . number_format(15 * 1.21, 2, ",", ".") . " €";
echo "Pantalón: " . number_format(32.5 * 1.21, 2, ",", ".") . " €";
echo "Zapatillas: " . number_format(59.9 * 1.21, 2, ",", ".") . " €";
```

Funciona. Pero si mañana cambia el IVA, tenéis que tocar tres líneas… o trescientas, en una tienda de verdad. Y en alguna os vais a equivocar. Con una función:

01-por-que-funciones.php — php-paso-a-paso/clase10 — Visual Studio Code

01-por-que-funciones.php

```php
<?php
function precioConIva($precio)
{
    return number_format($precio * 1.21, 2, ",", ".") . " €";
}

echo "Camiseta: " . precioConIva(15);
echo "Pantalón: " . precioConIva(32.5);
echo "Zapatillas: " . precioConIva(59.9);
```

Ahora el cálculo está **en un solo sitio**. Los programadores lo llaman principio **DRY** (*Don't Repeat Yourself*, «no te repitas»).

📄 Ejemplo: `01-por-que-funciones.php`

Las partes de una función

Esquema

```
palabra       nombre        parámetro
reservada       |              |
   |            ▼              ▼
function precioConIva($precio)
{
    return $precio * 1.21;    ⟵ lo que la función DEVUELVE
}

precioConIva(15);    ⟵ LLAMADA: aquí se ejecuta, con $precio = 15
```



💡 Escribir la función **no la ejecuta**. Solo la deja preparada. Se ejecuta cuando la **llamáis** por su nombre con paréntesis.



CLASE 10 · PUNTO 2

⚠️ La gran frontera: echo frente a return

Este es el concepto que más cuesta al principio. Prestad atención.

- Una función con `echo` es una **impresora**: saca el resultado por pantalla y ahí se queda. No podéis hacer nada más con él.
- Una función con `return` es una **calculadora**: os **devuelve** el resultado para que decidáis qué hacer con él: guardarlo, sumarlo, compararlo o mostrarlo.

02-echo-vs-return.php — php-paso-a-paso/clase10 — Visual Studio Code

02-echo-vs-return.php

```php
<?php
function sumarYMostrar($a, $b)
{
    echo $a + $b;          // impresora
}

function sumar($a, $b)
{
    return $a + $b;        // calculadora
}

sumarYMostrar(2, 3);              // muestra 5, y ya está

$total = sumar(2, 3);             // guarda 5 en $total
$doble = sumar($total, $total);   // y lo reutiliza: 10
```

¿Y si intentáis guardar lo que da la impresora? Obtenéis `NULL`: la función no ha devuelto nada.

📄 Ejemplo: `02-echo-vs-return.php`



🎯 **Mi recomendación:** que vuestras funciones **devuelvan** con `return` y que el `echo` lo haga quien las llama. Así la misma función os sirve para mostrar en una página, guardar en la base de datos o enviar por correo.

Un detalle más: en cuanto PHP llega a un `return`, **sale de la función**. Lo que haya debajo no se ejecuta.



CLASE 10 · PUNTO 3

📘 Parámetros: valores por defecto y argumentos con nombre

Valores por defecto

Un parámetro puede tener un valor por defecto: si al llamar a la función no lo pasáis, se usa ese [2].

03-parametros.php — php-paso-a-paso/clase10 — Visual Studio Code

03-parametros.php

```php
<?php
function calcularIva($importe, $tasa = 0.21)
{
    return $importe * $tasa;
}

echo calcularIva(100);          // 21  → usa el 21 %
echo calcularIva(100, 0.10);    // 10  → usa el 10 %
```



⚠️ Los parámetros con valor por defecto van **al final**. Primero los obligatorios, después los opcionales.

Argumentos con nombre

Desde PHP 8 podéis indicar **el nombre** del parámetro al llamar a la función. Así se entiende qué es cada dato y el orden deja de importar [2]:

03-parametros.php — php-paso-a-paso/clase10 — Visual Studio Code

03-parametros.php

```php
<?php
echo calcularIva(tasa: 0.04, importe: 50);   // 2
```

Donde más lucen es cuando hay varios parámetros opcionales y solo queréis cambiar uno:

03-parametros.php — php-paso-a-paso/clase10 — Visual Studio Code

03-parametros.php

```php
<?php
function saludar($nombre, $saludo = "Hola", $signo = "!")
{
    return "{$saludo}, {$nombre}{$signo}";
}

echo saludar("Ana");                  // Hola, Ana!
echo saludar("Ana", signo: "...");    // Hola, Ana...  ← nos saltamos $saludo
```

📄 Ejemplo: `03-parametros.php`



Podéis indicar el **tipo** de cada parámetro y el tipo del valor que devuelve la función [3]. Es como poner una etiqueta en la puerta: «aquí solo entran números enteros».

04-tipos.php — php-paso-a-paso/clase10 — Visual Studio Code

04-tipos.php

```php
<?php
declare(strict_types=1);

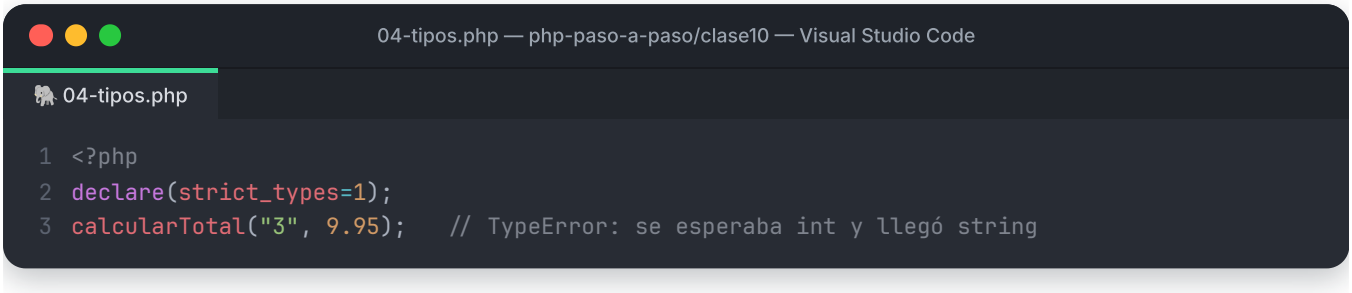
function calcularTotal(int $cantidad, float $precio): float
{
    return $cantidad * $precio;
}
```

Leedlo así: «`calcularTotal` recibe un entero y un decimal, y devuelve un decimal».

Tipo de retorno	Significa	Ejemplo
`: int`, `: float`, `: string`	Devuelve un número o un texto	`function sumar(int $a, int $b): int`
`: bool`	Devuelve `true` o `false`: preguntas de sí o no	`function esMayorDeEdad(int $edad): bool`
`: array`	Devuelve un array	`function obtenerNotas(): array`
`: void`	**No devuelve nada**; solo hace algo	`function mostrarLinea(string $t): void`

¿Y qué es `declare(strict_types=1)`?

Sin esa línea, PHP es «comprensivo»: si una función espera un `int` y le pasáis el texto `"3"`, lo convierte sin decir nada. Con ella, PHP se vuelve **estricto** y se para con un error `TypeError` [3].



¿Por qué quiero que lo uséis? Porque prefiero que el programa **falle en vuestro ordenador mientras practicáis** a que haga un cálculo raro sin avisar el día que esté cobrando a clientes de verdad.

Regla	
Dónde se escribe	Como **primera instrucción** del archivo, justo después de `<?php`
A qué afecta	A las llamadas a funciones hechas **en ese archivo**
Excepción útil	Un `int` sí puede entrar donde se espera un `float` (un 3 es un 3.0)



🎯 A partir de esta clase, todos los ejemplos empiezan con `declare(strict_types=1);`. Acostumbraos a escribirlo.

📄 Ejemplo: `04-tipos.php`. Al final tiene una línea comentada que provoca el `TypeError`: quitadle las `//` y leed el mensaje.



CLASE 10 · PUNTO 5

🔍 Ámbito: cada función tiene su habitación

Pensad en cada función como una **habitación con la puerta cerrada**. Las variables que creáis dentro se quedan dentro, y desde dentro no se ven las de fuera [4].

05-ambito.php — php-paso-a-paso/clase10 — Visual Studio Code

05-ambito.php

```php
<?php
$mensaje = "Soy de fuera";

function probarAmbito()
{
    $mensaje = "Soy de dentro";   // es OTRA variable, aunque se llame igual
    return $mensaje;
}

echo probarAmbito();   // Soy de dentro
echo $mensaje;         // Soy de fuera (no se ha tocado)
```

¿Y si la función necesita un dato de fuera? Se lo **pasáis por la puerta**, como parámetro:

05-ambito.php — php-paso-a-paso/clase10 — Visual Studio Code

05-ambito.php

```php
<?php
function saludar(string $nombre): string
{
    return "Hola, {$nombre}";
}

$usuario = "Pedro";
echo saludar($usuario);
```

📄 Ejemplo: `05-ambito.php`

Lejos de ser una molestia, esto es una gran ventaja: una función no puede estropear por accidente las variables del resto del programa.



💡 Veréis por internet la palabra `global` para «colarse» en la habitación. Funciona, pero os pido que no la uséis: hace el código difícil de seguir. Entrada por parámetros y salida por `return`.



CLASE 10 · PUNTO 6

😮 Funciones flecha: un primer vistazo

Esto es solo una presentación. Lo usaréis de verdad con los arrays en la clase 11 y, sobre todo, en Laravel.

Una **función flecha** es una función pequeñita, de **una sola expresión**, que se escribe con `fn` y `⇒` [5]:

06-funciones-flecha.php — php-paso-a-paso/clase10 — Visual Studio Code

06-funciones-flecha.php

```php
<?php
$doble = fn($n) ⇒ $n * 2;
echo $doble(5);   // 10
```

¿Para qué sirve algo así? Para pasársela a otra función y decirle «haz esto con cada elemento»:

06-funciones-flecha.php — php-paso-a-paso/clase10 — Visual Studio Code

06-funciones-flecha.php

```php
<?php
$precios = [10, 25, 40];
$conIva = array_map(fn($p) ⇒ $p * 1.21, $precios);
// [12.1, 30.25, 48.4]
```

📄 Ejemplo: `06-funciones-flecha.php`

Si ahora os parece raro, tranquilos. Es normal. Quedaos con la idea y lo retomamos más adelante.



CLASE 10 · PUNTO 7

👍 Cómo escribir buenas funciones

Consejo	Ejemplo
Una función, una tarea. Si para describirla necesitáis decir «y», probablemente son dos	`calcularTotal()` y `mostrarTicket()`, no `calcularYMostrarTicket()`
Nombre con verbo, en *camelCase*, que diga lo que hace	`calcularMedia()`, `esValido()`, `obtenerUsuario()`
Las que devuelven `bool`, como **pregunta**	`esPar()`, `tieneStock()`, `estaVacio()`
Cortas. Si no cabe en la pantalla, dividirla	
Devolved con `return`; el `echo`, fuera	
Tipos en parámetros y retorno, y `strict_types` arriba	`function esPar(int $n): bool`

El orden de un programa

Ahora que tenéis funciones, os pido que ordenéis **todos** vuestros programas siempre igual. Así, cuando abráis un archivo (vuestro o de un compañero), sabréis dónde está cada cosa sin buscar:

Orden	Parte	Qué va aquí	Ejemplo
1	**Funciones**	Todas las funciones del archivo	`function precioFinal(...)`
2	**Inicialización**	Las variables con su valor de partida	`$errores = [];`, `$total = 0;`
3	**Entradas**	Los datos que llegan de fuera: formularios, URL, sesión, ficheros	`$nombre = $_POST["nombre"] ?? "";`
4	**Procesamiento**	Los cálculos y las decisiones	validar, sumar, guardar...
5	**Salidas**	Lo que se muestra al usuario	`echo` o el HTML

ejemplo.php — php-paso-a-paso/clase10 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

function precioConIva(float $precio): float
{
    return round($precio * 1.21, 2);
}

$unidades = 3;

$precio = (float) ($_GET["precio"] ?? 10);

$total = precioConIva($precio) * $unidades;

echo "Total: {$total} €";
```

Fijaos en que no hace falta escribir comentarios con los nombres de cada parte: basta con **respetar el orden** y separar los bloques con una línea en blanco. En ejemplos de tres líneas no tiene sentido, pero en cuanto el programa crece, lo agradeceréis.



💡 Más adelante, cuando lleguen las **clases** de la programación orientada a objetos, irán todavía antes que las funciones.



CLASE 10 · PUNTO 8

🚀 Ejercicios

Empezad todos los archivos con `declare(strict_types=1);` y poned tipos a los parámetros y al retorno.

- ☐ **01 · Áreas.** `areaRectangulo(float $base, float $altura)` y `areaCirculo(float $radio)`, redondeada a 2 decimales. Pista: el número π está en la constante `M_PI`. → `ej01-area.php`
- ☐ **02 · Calificación.** `calificacion(float $nota): string` que devuelva Sobresaliente, Notable, Aprobado, Suspenso o «Nota no válida» si no está entre 0 y 10. Probadla con un `foreach`. → `ej02-calificacion.php`
- ☐ **03 · Precio final.** `precioFinal(float $precio, float $descuento = 0, float $iva = 0.21)` y otra función `formatoEuros()` que lo muestre como «121,00 €». Llamadla usando argumentos con nombre. → `ej03-precio-final.php`
- ☐ **04 · ¿Es par?** `esPar(int $numero): bool`. Usadla en un bucle del 1 al 10. → `ej04-es-par.php`
- ☐ **05 · Media.** `calcularMedia(array $notas): float`. ¿Qué debe devolver si el array está vacío? (Pensadlo: no se puede dividir entre cero). → `ej05-media.php`



🏆 **Reto:** volved al ejercicio de la entrada de cine de la clase 8 y convertidlo en una función `precioEntrada(int $edad, bool $esMiercoles): int`. ¿A que ahora es mucho más fácil probar todos los casos?



CLASE 10 · PUNTO 9

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Para qué	Escribir el código una vez y reutilizarlo (DRY)
`return`	Devuelve el resultado y sale de la función. Mejor que `echo` dentro
Por defecto	`$tasa = 0.21`: los opcionales, al final
Con nombre	`calcularIva(tasa: 0.10, importe: 50)`: el orden da igual
Tipos	`function sumar(int $a, int $b): int`; `void` si no devuelve nada
`strict_types`	Primera línea del archivo; si el tipo no coincide, `TypeError`
Ámbito	Lo de dentro se queda dentro: los datos entran por parámetros
Flecha	`fn($x) => $x * 2`: funciones de una línea (las veremos más)
Orden	Funciones → inicialización → entradas → procesamiento → salidas

📚 Fuentes y documentación oficial

1. Manual de PHP: funciones definidas por el usuario
2. Manual de PHP: argumentos de funciones (valores por defecto y argumentos con nombre)
3. Manual de PHP: declaraciones de tipo
4. Manual de PHP: ámbito de las variables
5. Manual de PHP: funciones flecha